

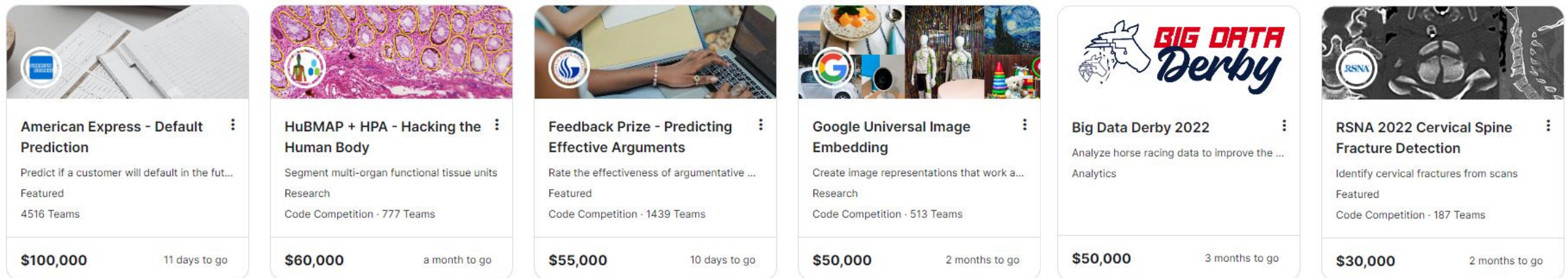
---

# KAGGLE COMPETITION

---

# Kaggle Competition

Kaggle is a public competition platform for machine learning where companies upload their data and invite participants from all over the world to help build models.



<https://www.kaggle.com/competitions>

- The team who builds the most accurate model receive awards.
- Gain hands-on experience in constructing machine learning models.

# Kaggle Competition



<https://web.archive.org/web/20090924184639/http://www.netflixprize.com/community/viewtopic.php?id=1537>

# kaggle

**Netflix Prize:** A Kaggle competition for the best movie recommendation system for predicting user ratings for films, based on previous ratings.

- Grand **prize of \$1,000,000 USD** was awarded (2009).
- Winners system had RMSE of 0.8567 on the testing set. Was a **10.06% improvement** over prior system.

# Kaggle Competition

Featured Prediction Competition

American Express - Default Prediction

Predict if a customer will default in the future

American Express

 · 4,586 teams · 7 days to go

\$100,000

Prize Money

Overview

Data

Code

Discussion

Leaderboard

Rules

Join Competition

...

Leaderboard

Public

Private

Raw Data

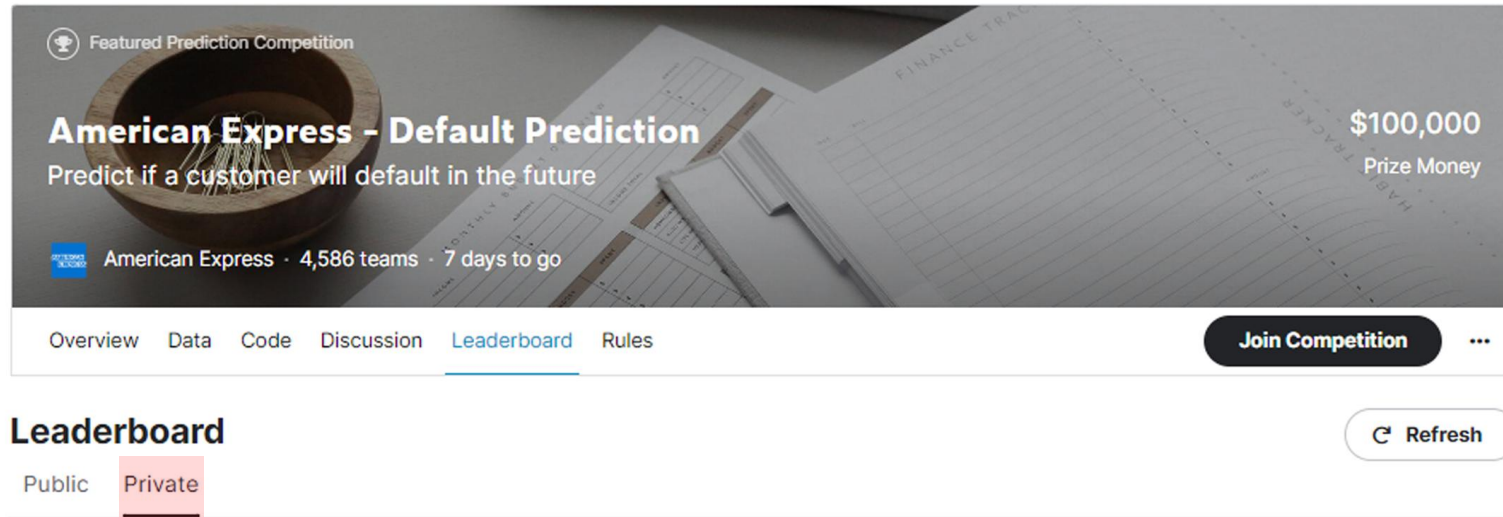
Refresh

**Public Leaderboard:**  
Score shown throughout the whole duration of the competition.

Prize Contenders

| # | Team                  | Members |  | Score | Entries | Last | Code |
|---|-----------------------|---------|--|-------|---------|------|------|
| 1 | Young for you         |         |  | 0.802 | 246     | 17h  |      |
| 2 | VISA                  |         |  | 0.801 | 358     | 1h   |      |
| 3 | byDefault [JuneHomes] |         |  | 0.801 | 310     | 8h   |      |
| 4 | Luck solo gold please |         |  | 0.801 | 45      | 4d   |      |
| 5 | Mengfei Li            |         |  | 0.801 | 264     | 11h  |      |

# Kaggle Competition



Featured Prediction Competition

## American Express - Default Prediction

Predict if a customer will default in the future

\$100,000  
Prize Money

American Express · 4,586 teams · 7 days to go

Overview Data Code Discussion Leaderboard Rules

[Join Competition](#) ...

### Leaderboard

Public Private

[Refresh](#)

## Private Leaderboard:

Performance only shown at the end of the competition (after the due date)



The Private Leaderboard isn't available yet.  
The final ranks and medals will display here after the competition closes.

**The private leaderboard is what matters!**

# Kaggle Competition

## This years Kaggle Competition: Music Genre Classification

Develop the best possible ML system to predict the genres of music. The task is to classify the music tracks into one of ten genres based on the provided audio features.



# Overview

Tutorial going over relevant content for the in-class Kaggle competition (assignment 3)

## Topics Covered:

- **Exploratory Data Analysis**
- **Data Pre-processing**
- **Machine Learning Modelling**
- **Model Evaluation and Validation**
- **Assignment Tips**



# Exploratory Data Analysis

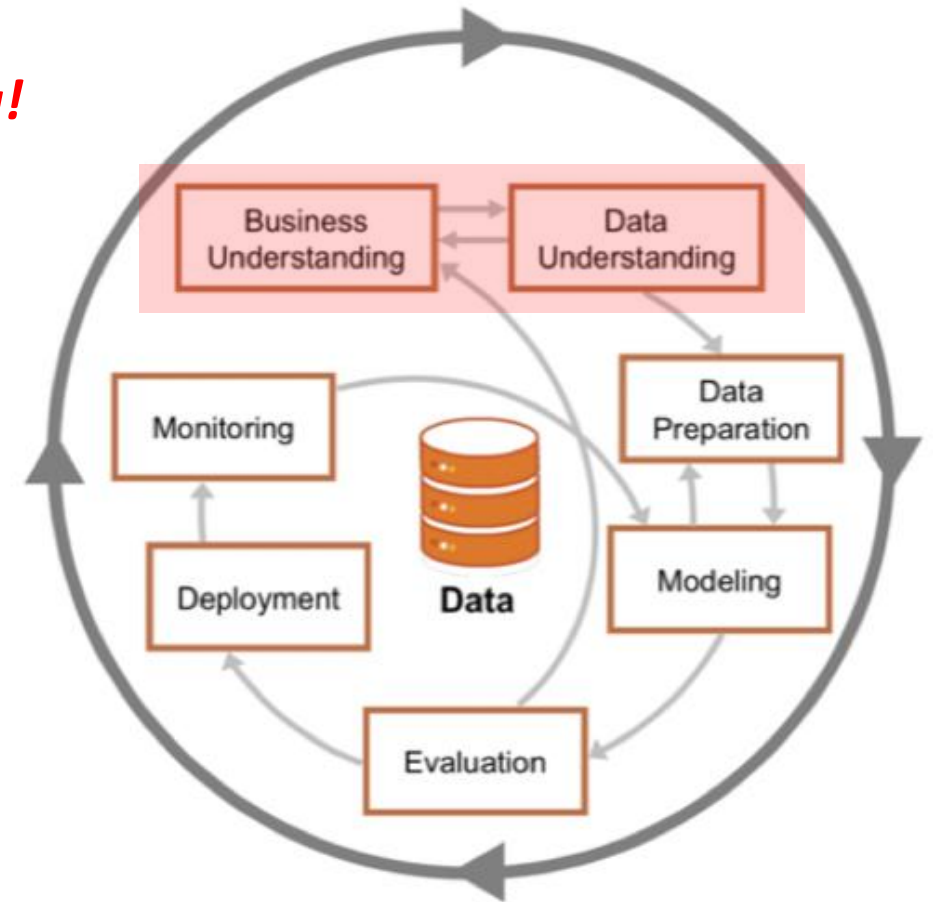
# Business and Data Understanding

*Before you do anything, you need to understand you data!*

- How many instances are in the dataset?
- How many features are in the dataset?
- What does each of the features represent?

Kaggle Competitions are primarily won by good data pre-processing and data transformation. Need business and data understanding to inform our decisions.

Last year all the top submissions had reports which showed significant insight into the given data.



**CRISP-DM Process**

# Kaggle Competition: Music Genre Classification

## Three Example Instances:

| 1 | artist_name   | track_name                 | track_id               | popularity | acousticness | danceability | duration_ms |
|---|---------------|----------------------------|------------------------|------------|--------------|--------------|-------------|
| 2 | B.o.B         | John Doe (feat. Priscilla) | 5T2AwbeUDXYbhjiDrD0s3e | 48         | 0.239        | 0.722        | 212006      |
| 3 | Prince Royce  | Back It Up                 | 0kCl6Aw5ikVtyQlvF4PwdO | 49         | 0.0895       | 0.736        | 200936      |
| 4 | Tamar Braxton | Let Me Know                | 28c4nfBHdb3xviamRCEsle | 47         | 0.0958       | 0.55         | -1          |

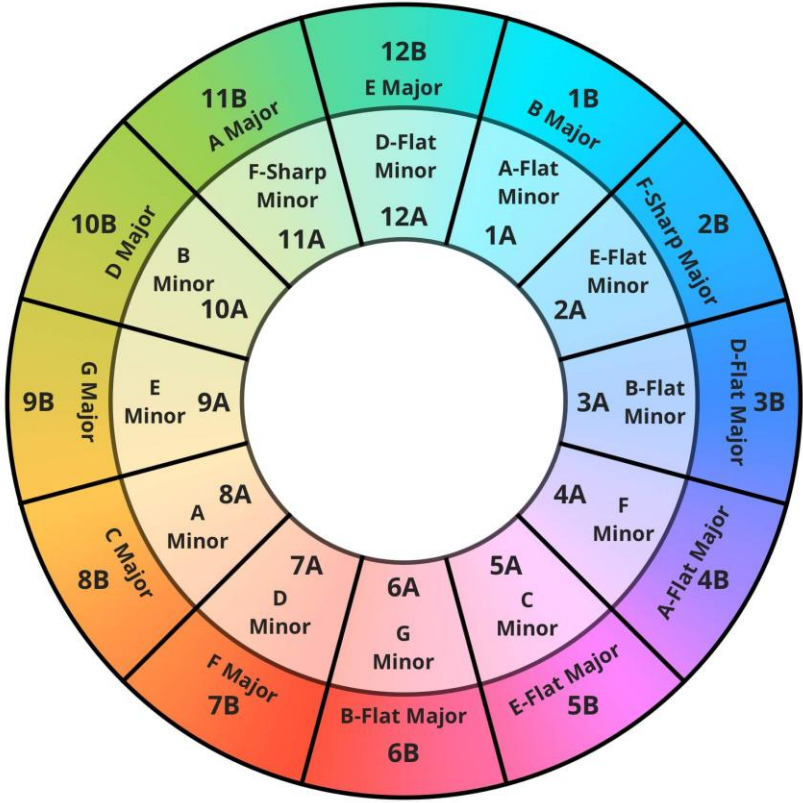
...

| energy | instrumentalness | key | liveness | loudness | mode  | speechiness | tempo   | time_signature | valence | genre |
|--------|------------------|-----|----------|----------|-------|-------------|---------|----------------|---------|-------|
| 0.697  | 0                | A#  | 0.062    | -7.083   | Major | 0.0404      | 120.132 | 4-Apr          | 0.487   | Dance |
| 0.743  | 0                | C#  | 0.0482   | -7.437   | Major | 0.0758      | ?       | 4-Apr          | 0.799   | Dance |
| 0.667  | 0                | D   | 0.138    | -5.362   | Major | 0.0346      | 78.019  | 4-Apr          | 0.433   | Dance |

...

Key and Mode features may requires domain knowledge to properly encode the relationship in a way the model will understand.

- You aren't always given all the information about the data. Consult subject matter experts (SME) and do research!



# Summary Statistics

Summarize a set of observations in order to communicate the largest amount of information as simply as possible.

- **Location:** Mean, Median, Mode
- **Spread:** Standard Deviation, Variance, Range
- **Shape:** Features Skew, Class distribution
- **Dependence:** Pearson's Correlation
- **Other:** Missing Values, Number of Features, Instances ...

**Don't reinvent the wheel!** Libraries for Summary Statistics:

**NumPy:** <https://numpy.org/doc/>

**Pandas:** <https://pandas.pydata.org/docs/>

**Statistics:** <https://docs.python.org/3/library/statistics.html>



# Helpful Functions

## Syntax – Creating DataFrames

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

```
df = pd.DataFrame(
    {"a" : [4 ,5, 6],
     "b" : [7, 8, 9],
     "c" : [10, 11, 12]},
    index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

|   | a | b | c |
|---|---|---|---|
| n | 1 | 4 | 7 |
| d | 2 | 5 | 8 |
| e | 2 | 6 | 9 |

```
df = pd.DataFrame(
    {"a" : [4 ,5, 6],
     "b" : [7, 8, 9],
     "c" : [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [('d',1),('d',2),('e',2)],
        names=['n','v']))
```

Create DataFrame with a MultiIndex

## Reshaping Data – Change the layout of a data set

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**pd.melt(df)**  
Gather columns into rows.

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**df.pivot(columns='var', values='val')**  
Spread rows into columns.

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**pd.concat([df1,df2])**  
Append rows of DataFrames

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**pd.concat([df1,df2], axis=1)**  
Append columns of DataFrames

**df.sort\_values('mpg')**  
Order rows by values of a column (low to high).

**df.sort\_values('mpg', ascending=False)**  
Order rows by values of a column (high to low).

**df.rename(columns = {'y':'year'})**  
Rename the columns of a DataFrame

**df.sort\_index()**  
Sort the index of a DataFrame

**df.reset\_index()**  
Reset index of DataFrame to row numbers, moving index to columns.

**df.drop(columns=['Length', 'Height'])**  
Drop columns from DataFrame

## Subset Observations (Rows)

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**df[df.Length > 7]**  
Extract rows that meet logical criteria.

**df.drop\_duplicates()**  
Remove duplicate rows (only considers columns).

**df.head(n)**  
Select first n rows.

**df.tail(n)**  
Select last n rows.

**df.sample(frac=0.5)**  
Randomly select fraction of rows.

**df.sample(n=10)**  
Randomly select n rows.

**df.iloc[10:20]**  
Select rows by position.

**df.nlargest(n, 'value')**  
Select and order top n entries.

**df.nsmallest(n, 'value')**  
Select and order bottom n entries.

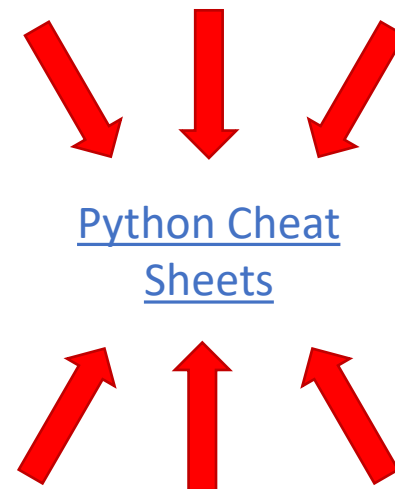
## Subset Variables (Columns)

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

**df[['width', 'length', 'species']]**  
Select multiple columns with specific names.

**df['width'] or df.width**  
Select single column with specific name.

**df.filter(regex='regex')**  
Select columns whose name matches regular expression *regex*.

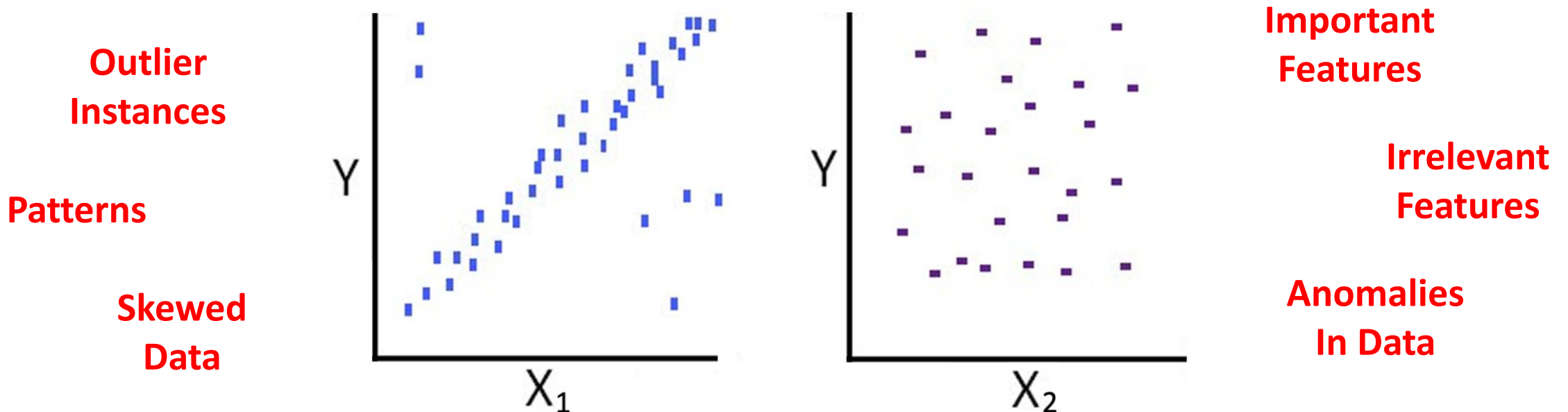


[Python Cheat  
Sheets](#)

# Basic Data Visualization

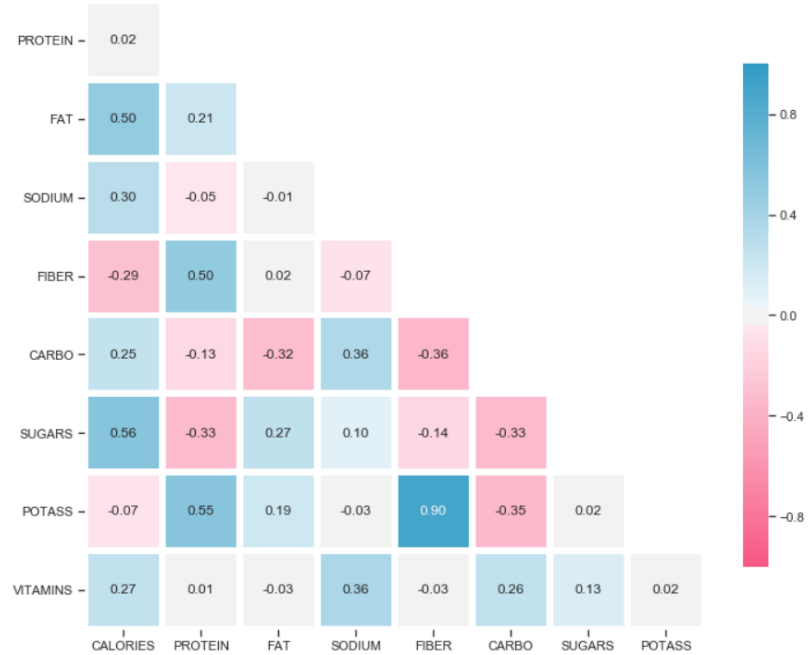
The graphical representation of information and data. Using visual elements like charts, graphs, and other data visualization tools to provide an accessible way to see and understand trends, and patterns in data.

Start with a simple scatterplot!

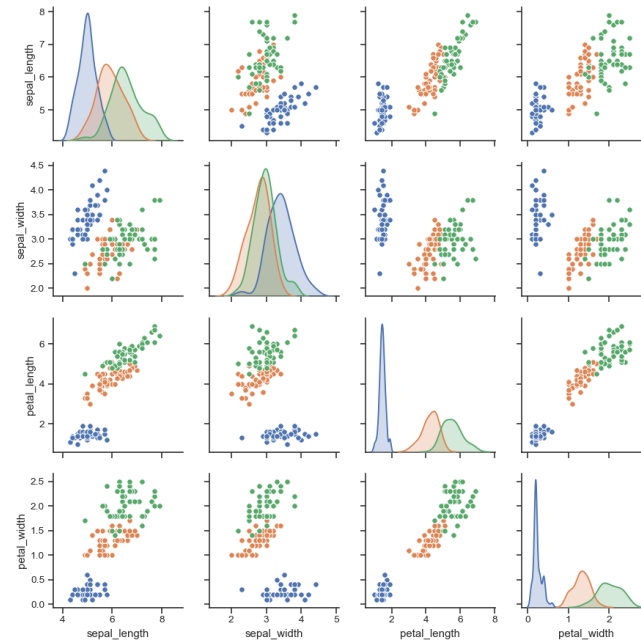


# Basic Data Visualization

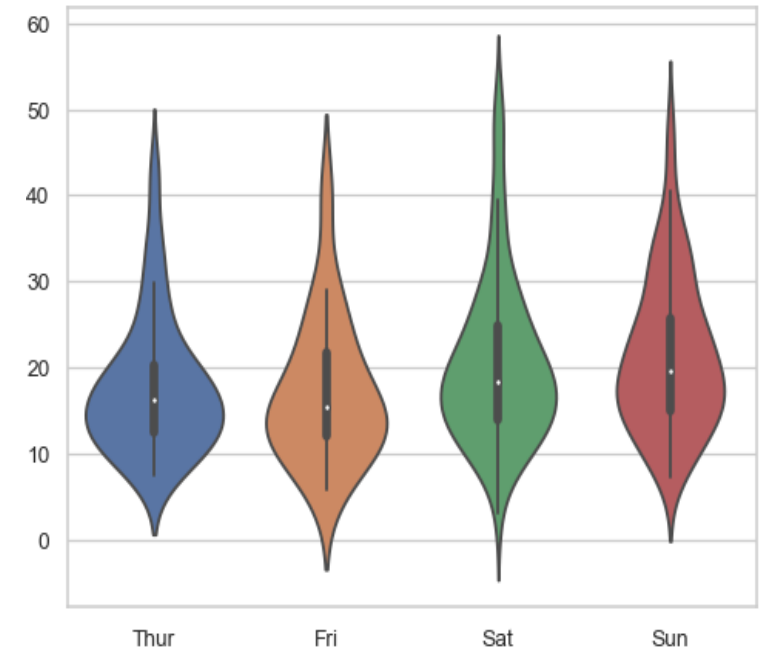
## Correlation Heatmap



## Scatter Matrix



## Violin Plot



[https://matplotlib.org/stable/plot\\_types/index.html](https://matplotlib.org/stable/plot_types/index.html)

<https://seaborn.pydata.org/examples/index.html>

# Identifying Missing Data

Missing data is not always easy to identify.

## Different Types of Missing Data:

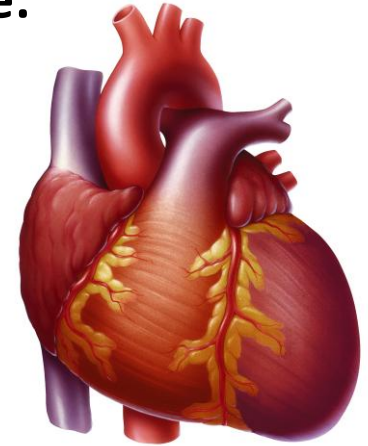
- Missing completely at random (MCAR)
- Missing at random (MAR)
- Missing not at random (MNAR)

**More about  
this in week 7**

## Solution to Missing Data:

1. **Remove** the missing data instances from the dataset.
2. Use **imputation** methods to substitute the missing values.
3. Use a **model which incorporates them**.

Example:



## Cardiovascular Dataset of Patients with Heart Condition

- Some patients had a Beats Per Minute (BPM) of 0
- Technician set some patients to 0 when they didn't record the BPM.

# Data Preprocessing

# Feature Selection

The process of selecting a subset of relevant features for use in model construction.

## Why should we perform Feature Selection?

1. Simplify the model and make it more interpretable
2. Reduce model training times
3. Remove redundant features
4. Remove irrelevant features

## How to perform Feature Selection?

- Use domain knowledge and expertise (Manual)
- Use feature selection techniques (Automatic)
  - Three types: Filter, Wrapper, and Embedded



**Titanic Dataset: Classify Live or Die**

# Feature Selection

The process of selecting a subset of relevant features for use in model construction.

| PassengerId | Survived | Name                                              | Sex    | Age  | SibSp | Parch | Ticket           | Fare    | Cabin | Embarked |
|-------------|----------|---------------------------------------------------|--------|------|-------|-------|------------------|---------|-------|----------|
| 1           | 0        | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     | A/5 21171        | 7.2500  | NaN   | S        |
| 2           | 1        | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     | PC 17599         | 71.2833 | C85   | C        |
| 3           | 1        | Heikkinen, Miss. Laina                            | female | 26.0 | 0     | 0     | STON/O2. 3101282 | 7.9250  | NaN   | S        |
| 4           | 1        | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     | 113803           | 53.1000 | C123  | S        |
| 5           | 0        | Allen, Mr. William Henry                          | male   | 35.0 | 0     | 0     | 373450           | 8.0500  | NaN   | S        |
| 6           | 0        | Moran, Mr. James                                  | male   | NaN  | 0     | 0     | 330877           | 8.4583  | NaN   | Q        |
| 7           | 0        | McCarthy, Mr. Timothy J                           | male   | 54.0 | 0     | 0     | 17463            | 51.8625 | E46   | S        |
| 8           | 0        | Palsson, Master. Gosta Leonard                    | male   | 2.0  | 3     | 1     | 349909           | 21.0750 | NaN   | S        |
| 9           | 1        | Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg) | female | 27.0 | 0     | 2     | 347742           | 11.1333 | NaN   | S        |



**Titanic Dataset: Classify Live or Die**

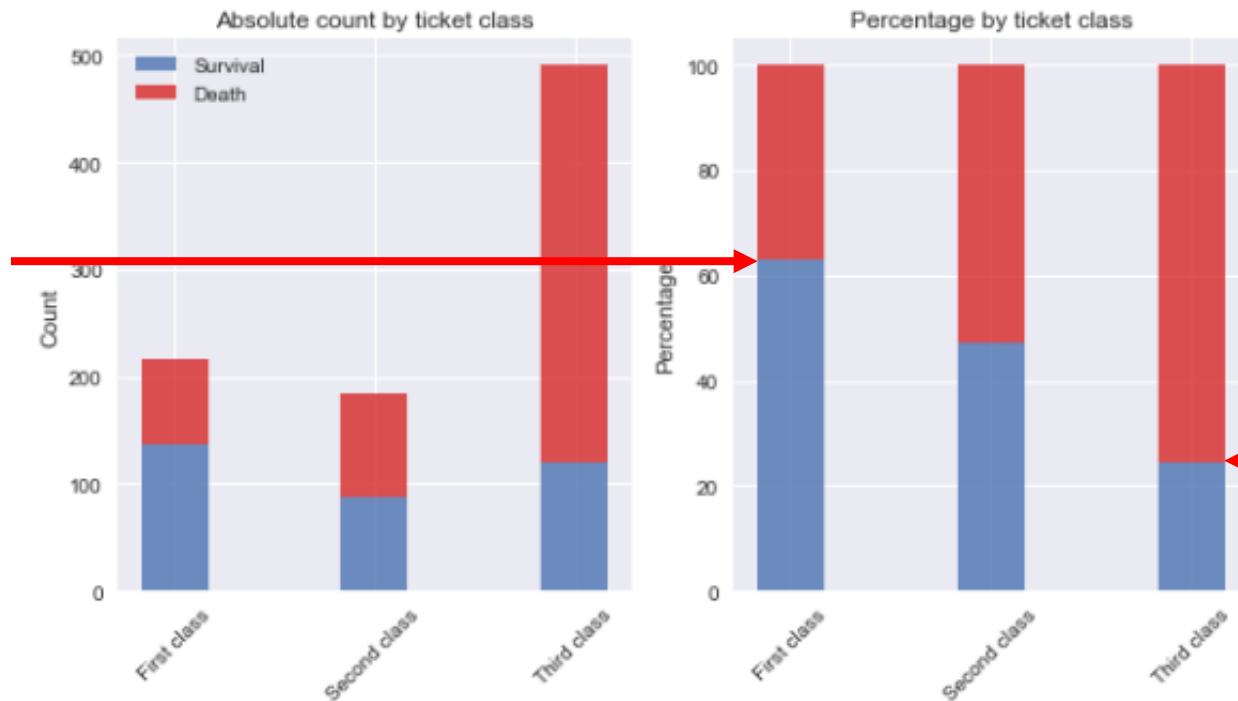
**Should we delete the Passenger ID ??**

# Feature Construction

**Passenger ID corresponds to the Ticket Class!**

- Manually create a new feature “Ticket Class” which is derived from the “Passenger ID”.

**First Class  
= 62.96%  
Survival Rate**



**Third Class  
= 24.23%  
Survival Rate**



# Data Encoding

Encode data in a representation the machine understands. Encoding should retain information.

## Ordinal Data, i.e. ordered categories:

- Ordinal Encoding
- Cyclical Encoding

## Nominal Data, i.e. unordered categories:

- Label Binarizing
- One-Hot Encoding
- Dummy Encoding

## Text Data, i.e. unstructured raw text:

- Word Statistics
- Bag-of-Words
- TF-IDF
- Word Embeddings

Hot, Hotter, Hottest



Date and Time



Fruit Type



Race, Gender



**Pros and Cons of Different  
Encoding Strategies?**

Emails, Webpages, etc.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae

**Do we always need to  
encode our data?**

# Imputation

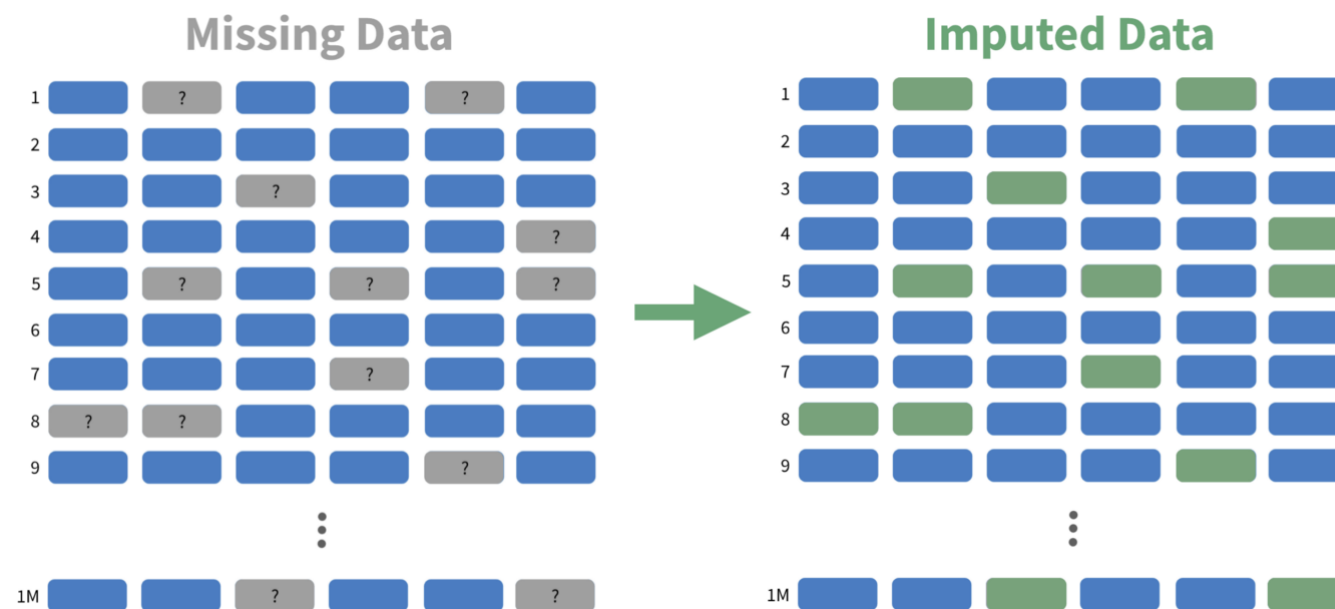
The process of **replacing missing data** with substituted values.

## Common Imputation Strategies:

- Mean Imputation
- Mode Imputation
- Hot Deck Imputation
- Cold Deck Imputation
- Nearest Neighbour Imputation
- ...

## Imputation vs Removal vs Nothing?

- Imputation runtime concerns
- Instance missing multiple features
- Model can handle missing values



<https://scikit-learn.org/stable/modules/impute.html>  
[https://en.wikipedia.org/wiki/Imputation\\_\(statistics\)](https://en.wikipedia.org/wiki/Imputation_(statistics))

# Machine Learning Modelling

# Selecting a Machine Learning Model

**Question:** So many options ... which machine learning model should be used?

**Answer:** It depends on your data and requirements.

**For tabulated data (*i.e.* our kaggle competition) some suggestions:**

- **Linear Models** – Logistic Regression, Elastic Net, GAMs, ...
- **Tree Ensembles** – Random Forest, Gradient Boosting, XGBoost, ...
- **Neural Networks** – Feed-Forward Neural Networks, TabNet, ...

**Side Note: For Computer Vision and Natural Language Processing:**

- **Neural Networks** - Convolutional Neural Networks and Transformers.

**Things to consider when choosing a model:**

- Training and inference runtime
- Time to implement algorithm/method
- Predictive performance of the model

# Hyper-Parameter Optimization

Parameters which **controls the settings of the algorithm**. Not directly learned by the algorithm/model, e.g. k in k-nearest neighbour.

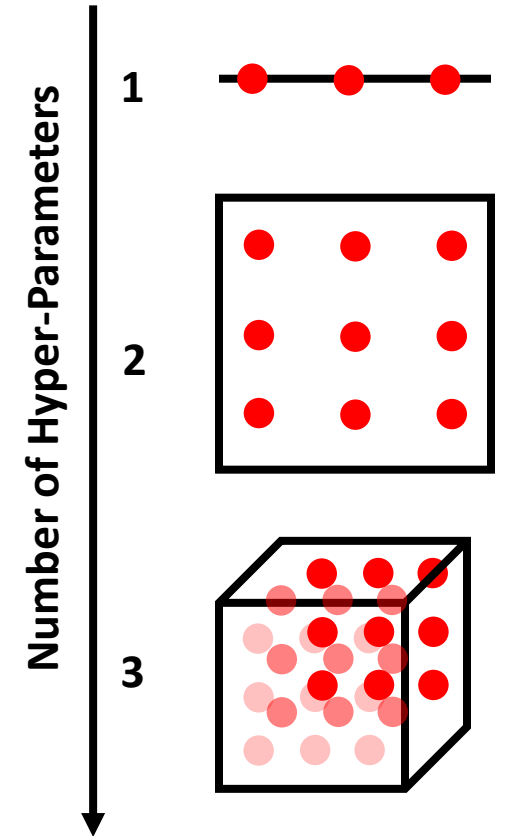
## How to select the best hyper-parameters for your model?

1. Start with the default used by the library.
2. Learn about you chosen machine learning model.
3. Identify which hyper-parameters (1 or 2) are most important.
4. Grid Search or Random Search.

## Practical Advice for Kaggle Competition:

Do better Hyper-parameter optimization techniques exist? **A: Yes.**

Do you have enough time and computational resources? **A: Probably not.**



# Bagging, Boosting, and Stacking

Ensembling approaches for **combine multiple machine learning techniques into one** model in order to improve the performance.

**Bagging:** Models learn independently from each other in parallel and then combined by taking the average prediction across models.

**Stacking:** Models learn independently from each other in parallel and then combined by using a meta-model to output the final prediction.

**Boosting:** Models learn sequentially in an adaptative way (a model depends on the previous ones) and combines them following a strategy.

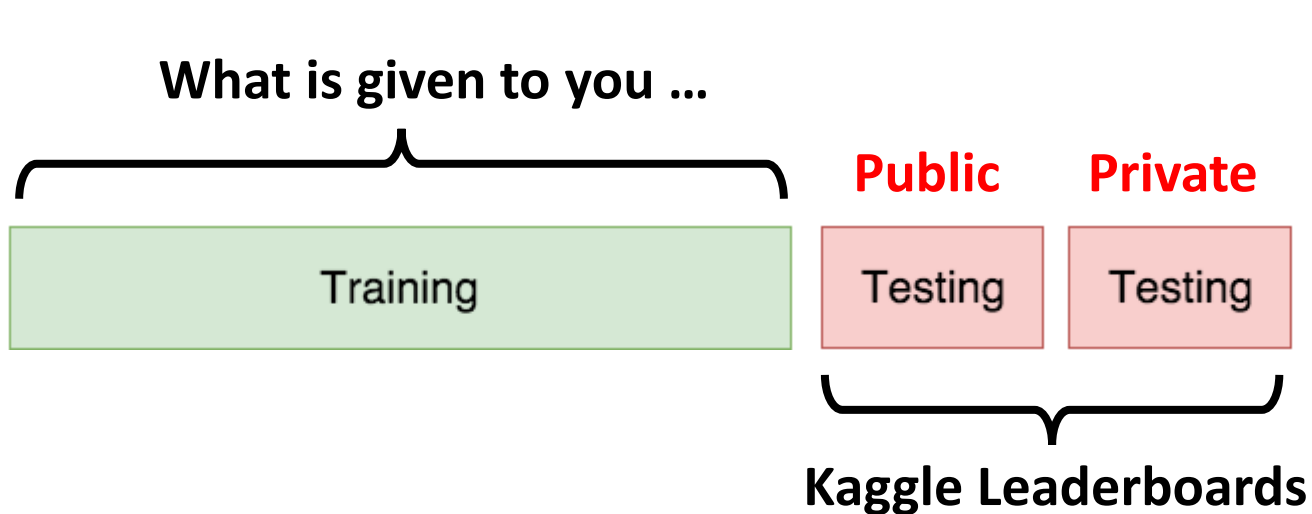
<https://towardsdatascience.com/ensemble-methods-bagging-boosting-and-stacking-c9214a10a205>

<https://scikit-learn.org/stable/modules/ensemble.html>

# Model Evaluation and Validation

# Cross Validation

Techniques for **assessing how a machine learning model will generalize to new unseen data**.  
Mainly used in settings where the goal is prediction, and one wants to estimate how accurately a predictive model will **perform in the real world**.



## Cross Validation Techniques:

- Training-Testing Split
- Training-Validation-Testing Split
- k-Fold Cross-Validation
- Stratified Cross-validation
- Leave-k-Out Cross-Validation
- Leave-One-Out Cross-Validation
- Random Sub-Sampling Validation

**Suggested Reading!**



<https://mlu-explain.github.io/train-test-validation/>  
[https://en.wikipedia.org/wiki/Cross-validation\\_\(statistics\)](https://en.wikipedia.org/wiki/Cross-validation_(statistics))

# Kaggle Competition Public Leaderboard

| #  | Team             | Members                                                                             | Score   | Entries | Last | Code |
|----|------------------|-------------------------------------------------------------------------------------|---------|---------|------|------|
| 1  | HowOrdinawy      |    | 0.99691 | 6       | 1y   |      |
| 2  | Shaun C.         |    | 0.99485 | 18      | 1y   |      |
| 3  | Dave from Chorus |    | 0.99205 | 10      | 1y   |      |
| 4  | Jefffffffffff    |    | 0.97470 | 6       | 1y   |      |
| 5  | New_ai           |    | 0.96411 | 28      | 1y   |      |
| 6  | PatttttttttttRY  |    | 0.93676 | 5       | 1y   |      |
| 7  | Maxwell Grigson  |    | 0.84441 | 8       | 1y   |      |
| 8  | FlyingAlpaca     |   | 0.80250 | 5       | 1y   |      |
| 9  | GaoYuan          |  | 0.80176 | 12      | 1y   |      |
| 10 | happydemic       |  | 0.78794 | 9       | 1y   |      |
| 11 | MurpheyWu        |  | 0.78455 | 37      | 1y   |      |
| 12 | Globlax          |  | 0.78308 | 5       | 1y   |      |

173

Teams

173

Competitors

1,591

Entries

Example of the **top**  
*public submissions*  
from last years Kaggle  
competition

# Kaggle Competition Private Leaderboard

| #  | Δ    | Team             | Members                                                                             | Score   | Entries | Last | Code |
|----|------|------------------|-------------------------------------------------------------------------------------|---------|---------|------|------|
| 1  | ▲ 7  | FlyingAlpaca     |    | 0.81030 | 5       | 1y   |      |
| 2  | ▲ 7  | GaoYuan          |    | 0.79840 | 12      | 1y   |      |
| 3  | ▲ 7  | happydemic       |    | 0.79098 | 9       | 1y   |      |
| 4  | ▲ 3  | Maxwell Grigson  |    | 0.78750 | 8       | 1y   |      |
| 5  | ▲ 6  | MurpheyWu        |    | 0.78439 | 37      | 1y   |      |
| 6  | ▼ 4  | Shaun C.         |    | 0.78386 | 18      | 1y   |      |
| 7  | ▲ 6  | eau              |    | 0.78113 | 14      | 1y   |      |
| 8  | ▲ 6  | DONDA            |   | 0.78045 | 9       | 1y   |      |
| 9  | ▲ 3  | Globlax          |  | 0.77113 | 5       | 1y   |      |
| 10 | ▼ 7  | Dave from Chorus |  | 0.74803 | 10      | 1y   |      |
| 11 | ▲ 4  | Bayden           |  | 0.73053 | 13      | 1y   |      |
| 12 | ▼ 11 | HowOrdinawy      |  | 0.72348 | 6       | 1y   |      |

Models which **overfitted the public leaderboard failed to generalize** their performance to the private leaderboard.

Largest Increase in Rank ▲ 32

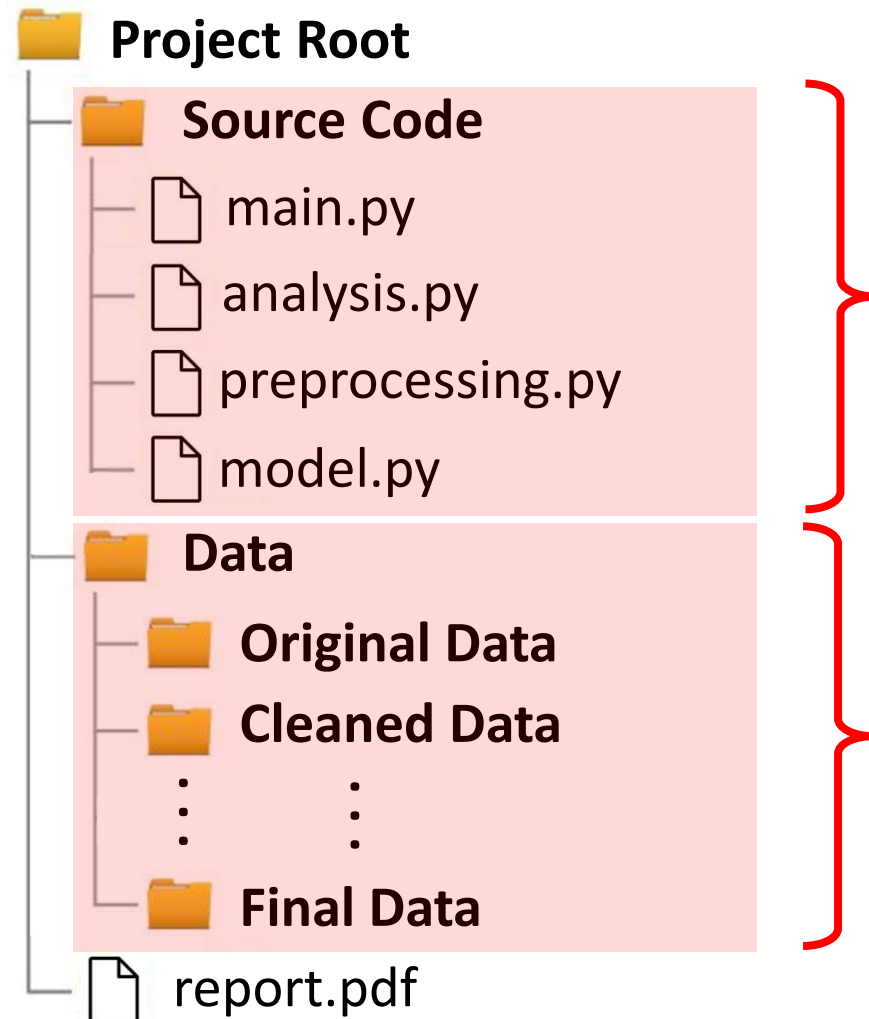
Largest Decrease in Rank ▼ 83

# Assignment Tips

# Project File Structure

## Suggested Project Structure:

- Separate the code logic from the data.
- Maintaining version control for the data



## Source Code

- Run code from main file, separate out the ML pipeline stages.

## Data Version Control

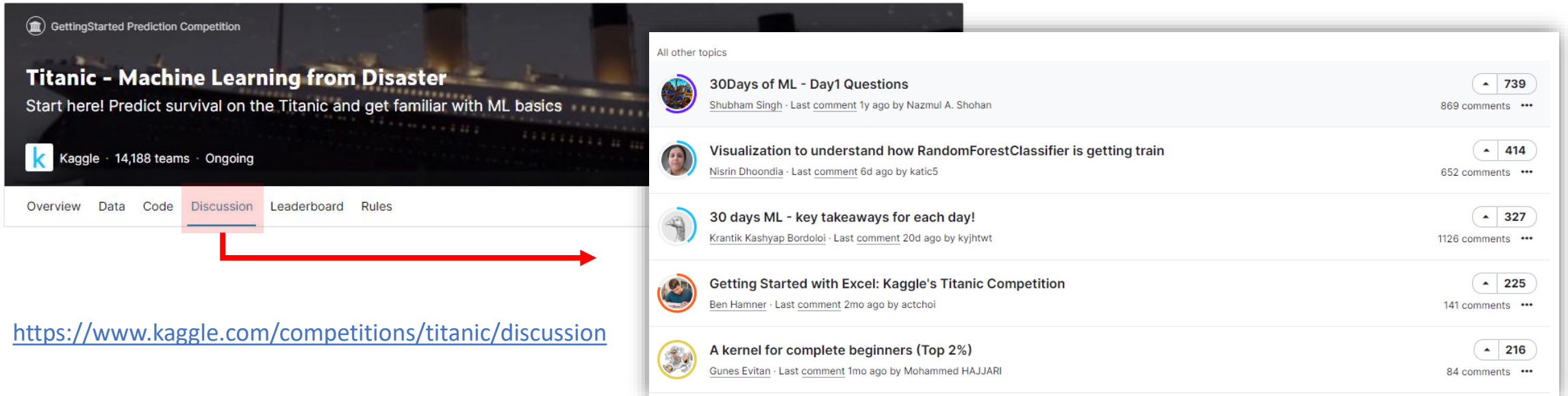
- Don't need to run pre-processing pipeline every time can just load data.

# Limitations of Kaggle Competitions

1. **Not all data science problems require machine learning.** Other skills such as SQL, business analytics, data visualization, and statistics is needed instead.
2. Constructing a model is only a **small fraction of a data scientist's daily work.**
3. Most of the models developed in Kaggle competitions are way **too complex to be deployed.** Maintaining a complex model is an engineering challenge.
4. The **performance is not the only measure of a successful** model. Other aspects such as the interpretability, model size, inference speed are often considered.
5. To get a high score on Kaggle, you will sometimes see people **utilize tricks that aren't available in real world data**, *e.g.* exploit data leakage.

# Closing Remarks

1. The report is what is marked. **Ensure you document all the hard work you are doing!**
2. To do well in the competition you need to **do your own research**. Get creative.
3. **Discuss your approach** with your peers (but beware of plagiarism) and tutors.
4. Read the **discussion sections of past Kaggle competitions**.



The screenshot shows the Kaggle Titanic competition page. The 'Discussion' tab is highlighted in the navigation bar. A red arrow points from the 'Discussion' tab to a list of discussion topics. The list includes:

- 30Days of ML - Day1 Questions** (739 comments)
- Visualization to understand how RandomForestClassifier is getting train** (414 comments)
- 30 days ML - key takeaways for each day!** (327 comments)
- Getting Started with Excel: Kaggle's Titanic Competition** (225 comments)
- A kernel for complete beginners (Top 2%)** (216 comments)

The URL <https://www.kaggle.com/competitions/titanic/discussion> is displayed below the navigation bar.

# Data Loading

Many **different storage** formats. Different Pros and Cons:

- Scalability
- Accessibility
- Latency
- Throughput
- Parallel Access

**Learn More in SWEN304**

**Kaggle competition – 10 .csv files (EASY!)**

## Example

Load the CSV into a DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```

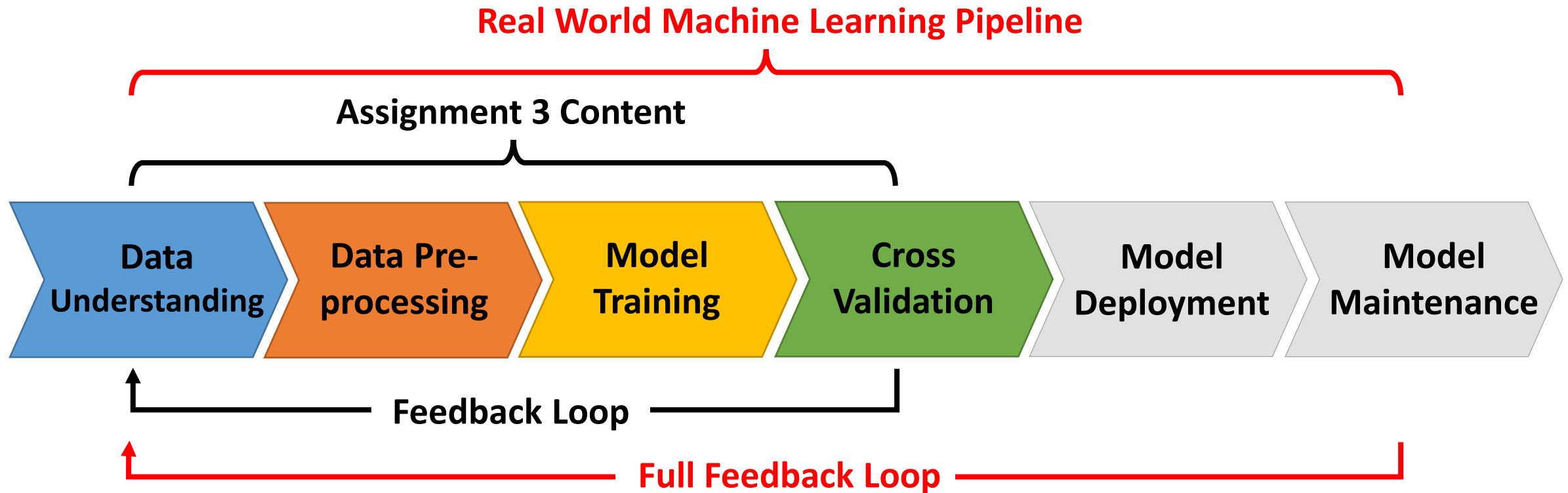
**Don't need to  
write a custom  
a data parser!**

| df1 |     |     |     |     | Result |     |     |     |     |
|-----|-----|-----|-----|-----|--------|-----|-----|-----|-----|
|     | A   | B   | C   | D   |        | A   | B   | C   | D   |
| 0   | A0  | B0  | C0  | D0  | 0      | A0  | B0  | C0  | D0  |
| 1   | A1  | B1  | C1  | D1  | 1      | A1  | B1  | C1  | D1  |
| 2   | A2  | B2  | C2  | D2  | 2      | A2  | B2  | C2  | D2  |
| 3   | A3  | B3  | C3  | D3  | 3      | A3  | B3  | C3  | D3  |
| df2 |     |     |     |     | 4      | A4  | B4  | C4  | D4  |
|     | A   | B   | C   | D   | 5      | A5  | B5  | C5  | D5  |
| 4   | A4  | B4  | C4  | D4  | 6      | A6  | B6  | C6  | D6  |
| 5   | A5  | B5  | C5  | D5  | 7      | A7  | B7  | C7  | D7  |
| 6   | A6  | B6  | C6  | D6  | 8      | A8  | B8  | C8  | D8  |
| 7   | A7  | B7  | C7  | D7  | 9      | A9  | B9  | C9  | D9  |
| df3 |     |     |     |     | 10     | A10 | B10 | C10 | D10 |
|     | A   | B   | C   | D   | 11     | A11 | B11 | C11 | D11 |
| 8   | A8  | B8  | C8  | D8  |        |     |     |     |     |
| 9   | A9  | B9  | C9  | D9  |        |     |     |     |     |
| 10  | A10 | B10 | C10 | D10 |        |     |     |     |     |
| 11  | A11 | B11 | C11 | D11 |        |     |     |     |     |

[https://pandas.pydata.org/docs/reference/api/pandas.read\\_csv.html](https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html)  
[https://pandas.pydata.org/docs/user\\_guide/merging.html](https://pandas.pydata.org/docs/user_guide/merging.html)

# Machine Learning Pipeline

Should be able to perform the whole machine learning pipeline end-to-end.

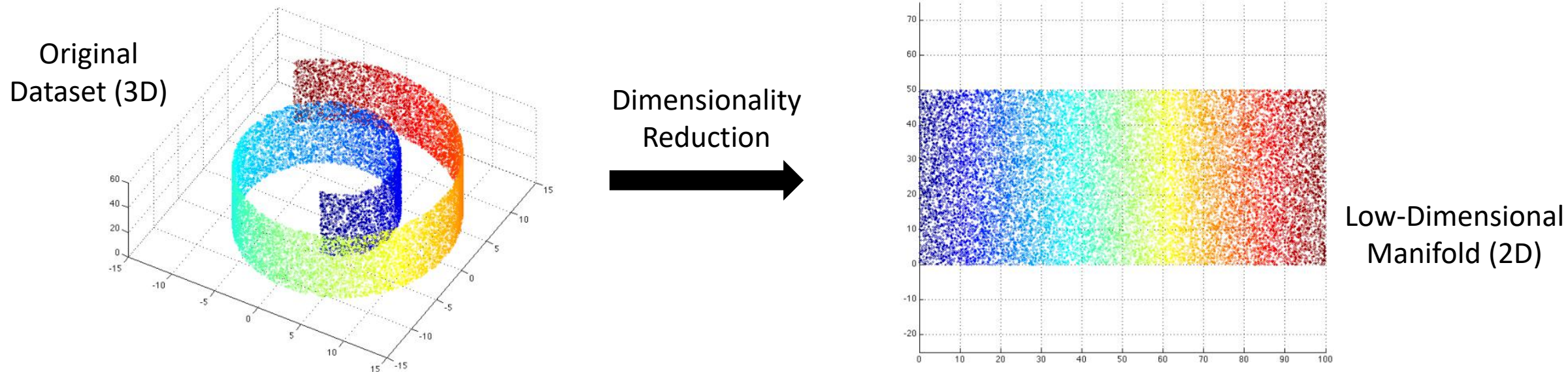


# Complex Data Visualization

## Non-Linear Dimensionality Reduction (NDR):

Mapping high-dimensional data into a low-dimensional space for visualization.

- **Manifold Hypothesis:** Many high-dimensional data sets that occur in the real world lies along low-dimensional manifolds inside that high-dimensional space.



# Complex Data Visualization

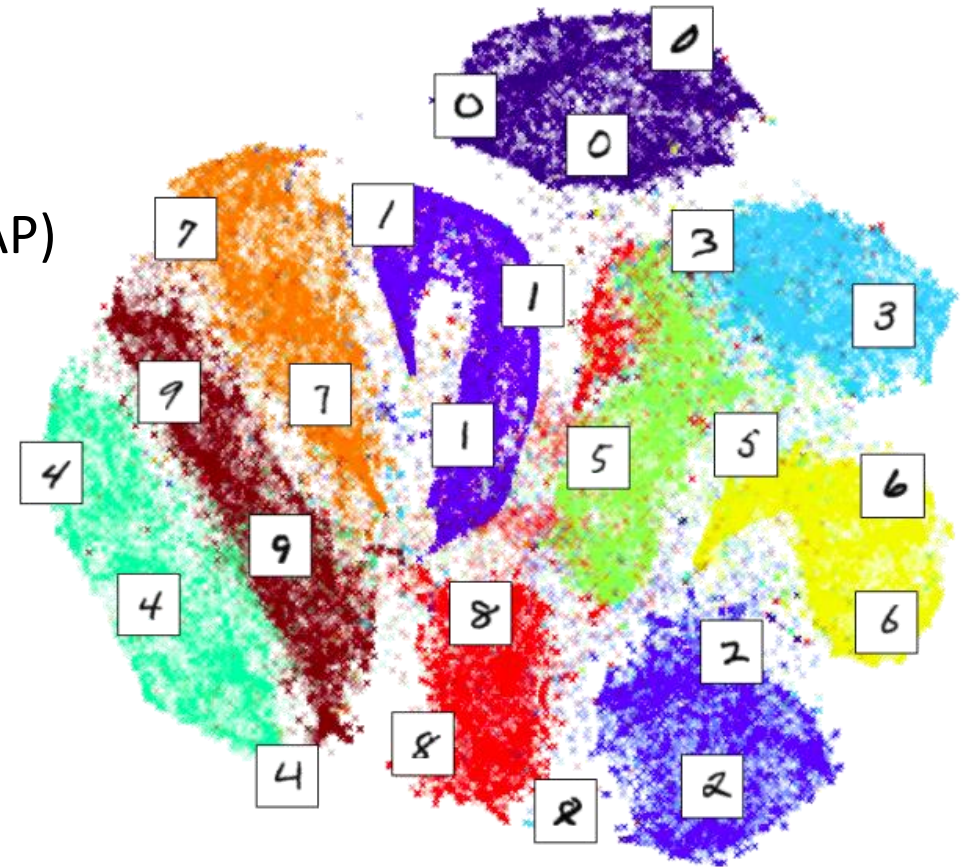
## Popular NDR Techniques:

- Uniform Manifold Approximation and Projection (UMAP)
- t-Distributed Stochastic Neighbour Embedding (t-SNE)

<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.manifold>

## Why should we perform NDR?

- Identifying **separation of classes**
- Identifying **overlapping classes**
- Setting preliminary expectations about the ML models performance



MNIST Character Recognition Dataset.  
Mapping  $28 \times 28 = 784$  dimensions down to 2.